GitHub Actions Event Triggers — Interview Notes

Core Concept

- An **event** is a specific activity in a repository that triggers a workflow run [1] [2]
- Defined in the workflow YAML using `on:` [3] [4]

1. push

When: Code is pushed to a branch or tag
Use cases: CI on every commit, auto-deploy on main
Key syntax:

```
on:
  push:
    branches: [main, develop]
    tags: ['v*']
    paths:
      - 'src/**'
      - '!src/docs/**'
```

Notes:

- Triggers on branch pushes and tag pushes
- `paths`/`paths-ignore` filter by file changes [3]
- `github.ref`, `github.sha` available in context [5]

2. pull_request

When: PR is opened, synced, reopened, closed, etc.
Use cases: PR validation, CI checks before merge
Key syntax:

```
on:
  pull_request:
    branches: [main]
    types: [opened, synchronize, reopened]
```

Activity types: `opened`, `edited`, `closed`, `reopened`, `synchronize`, `assigned`, etc.
Notes:

- Runs in the context of the **base branch** (safe for forks)
- `github.event.pull_request.*` gives PR details[5]
- Use `pull_request_target` for fork PRs needing secrets (more privileged)

3. workflow_dispatch

When: Manually triggered from GitHub UI or API
Use cases: Manual deployments, ad-hoc runs
Key syntax:

```
on:
  workflow_dispatch:
    inputs:
      environment:
        description: 'Environment to deploy'
        required: true
        default: 'staging'
```

Notes:

- Supports **inputs** for dynamic parameters[6]
- Great for debugging and manual releases

4. schedule (cron)

When: Scheduled time using POSIX cron syntax
Use cases: Daily cleanup, periodic tests, scheduled deployments
Key syntax:

```
on:
  schedule:
    - cron: '0 0 * * *'  # Midnight UTC daily
```

Notes:

- 5-field cron: `minute hour day month weekday`
- Runs in UTC timezone
- Minimum granularity: 5 minutes[3]

5. release

When: GitHub release events (published, created, prereleased, etc.)
Use cases: Auto-publish on release, versioned deployments
Key syntax:

```
on:
  release:
    types: [published]
```

Activity types: `published`, `prereleased`, `released`, `edited`, `deleted`

6. issues

When: Issues are opened, edited, assigned, closed, etc.
Use cases: Auto-labeling, notifications, issue bots
Key syntax:

```
on:
  issues:
    types: [opened, assigned]
```

Activity types: `opened`, `edited`, `assigned`, `closed`, `reopened`, `labeled`

7. issue_comment

When: Comment is added to an issue or PR
Use cases: `/deploy` commands, bot interactions
Key syntax:

```
on:
  issue_comment:
    types: [created]
```

Notes:

- `github.event.comment.body` contains the comment text
- Commonly used for command-triggered workflows

8. workflow_run

When: Another workflow completes (e.g., after CI finishes)
Use cases: Post-CI steps, security scanning after build
Key syntax:

```yaml
on:
  workflow_run:
    workflows: ["CI"]
    types: [completed]
```

Notes:

- Can access artifacts from the triggering workflow
- Useful for multi-workflow orchestration

9. workflow_call

When: A reusable workflow is called by another workflow
Use cases: Creating reusable/cross-repo workflows
Key syntax:

```yaml
on:
  workflow_call:
    inputs:
      environment:
        type: string
    secrets:
      BUILD_TOKEN:
        required: true
```

Notes:

- Must be in `.github/workflows/`
- Supports inputs and secrets passthrough

10. repository_dispatch

When: External service sends a webhook to GitHub
Use cases: Trigger from external CI, webhooks from other services
Key syntax:

```yaml
on:
  repository_dispatch:
    types: [custom-event]
```

Notes:

- Triggered via GitHub API: `POST /repos/{owner}/{repo}/dispatches`
- Custom event types you define

11. deployment / deployment_status

When: A deployment is created or its status changes
Use cases: Deploy review actions, rollback on failure
Key syntax:

```
on:
  deployment_status:
    types: [success, failure]
```

Notes:

- `deployment` = deployment created
- `deployment_status` = status updated (success/failure/in_progress)

12. pull_request_target

When: Same as `pull_request` but runs in **base branch** context
Use cases: PR checks from forks needing access to secrets
Key syntax:

```
on:
  pull_request_target:
    types: [opened, synchronize]
```

⚠ **Security warning:**

- Has write permissions by default
- Don't checkout untrusted PR code without caution
- More privileged than `pull_request`

13. check_run / check_suite

When: A check run or check suite is created/completed
Use cases: Custom check integration, status updates
Key syntax:

```
on:
  check_run:
    types: [completed]
```

14. create / delete

When: Branch or tag is created/deleted
Use cases: Auto-cleanup, notifications on tag creation
Key syntax:

```
on:
  create:
```

Notes:

- `github.ref_type` tells if it's `branch` or `tag`

15. fork

When: Repository is forked
Use cases: Notify on fork, update fork-related configs

16. gollum

When: Wiki page is created/edited
Use cases: Wiki change notifications

17. page_build

When: GitHub Pages build completes
Use cases: Notification after site build

18. registry_package / npm_package

When: Package is published to GitHub Packages / npm
Use cases: Package validation, downstream deployments

19. project / project_card / project_column

When: GitHub Projects activity occurs
Use cases: Project automation bots

20. watch

When: Someone stars/watches the repository
Use cases: Community engagement tracking

Quick Interview Cheat Sheet

Event	Trigger Condition	Common Use
`push`	Code pushed	CI on every commit [3]
`pull_request`	PR opened/updated	PR validation
`workflow_dispatch`	Manual trigger	Manual deploy [6]
`schedule`	Cron time	Daily tasks [3]
`release`	Release published	Auto-publish binaries
`issue_comment`	Comment on issue/PR	Bot commands
`pull_request_target`	PR from fork	Fork PR with secrets
`workflow_run`	Another workflow completes	Post-CI steps
`repository_dispatch`	External webhook	External CI integration
`deployment_status`	Deployment status changes	Deploy review

Key Interview Points

1. `push` **vs** `pull_request`: PR runs on base branch, safer for forks
2. `workflow_dispatch`: Only way to manually trigger from UI
3. `schedule`: Uses cron, runs in UTC, min 5-minute interval
4. `pull_request_target`: More privileged, use carefully with forks
5. `workflow_call`: Required for reusable workflows
6. `paths` **filtering**: Reduces unnecessary workflow runs
7. **Event payloads differ**: `github.event` content depends on event type [5]

⁂

1. https://docs.github.com/actions/using-workflows/events-that-trigger-workflows
2. https://docs.github.com/articles/getting-started-with-github-actions
3. https://www.actionsbyexample.com/event-triggers.html
4. https://docs.github.com/actions/using-workflows/workflow-syntax-for-github-actions
5. https://stackoverflow.com/questions/70104600/complete-list-of-github-actions-contexts
6. https://docs.github.com/actions/using-workflows/triggering-a-workflow
7. https://www.youtube.com/watch?v=ldRYf4-8AzM
8. https://github.com/marketplace/actions/trigger-workflow-and-wait
9. https://gist.github.com/devops-school/0bd2f0f20de20baf2f26526c2e0f733e
10. https://www.git-tower.com/blog/github-actions-events-and-triggers